Project: R&E - Surface Tension

A cybersteps project

Date: 14. November 2025

Participants: Gregor

Part 1: Subdomain Enumeration

Executive summary:

There is a list of 100 root domains, listed in the file 'domains.txt'. The mission is to discover as many valid subdomains as you can. This part tests the ability to explore and filter information effectively, as well as selecting and using the right tools for the job.
Only passive reconnaissance is allowed.

Execution:

I decided to use the tool theHarvester to look up discovered hosts. Because of the lack of different API-keys I decided to use lookup over virustotal, because I had an API-Key for that.

To automate it for every 100 given domains I decided to use python's subprocess module.

```
def run_theharvester(domain):
    cmd = [
        "theHarvester",
        "-d", domain,
        "-b", "virustotal",
    ]
    result = subprocess.run(cmd, capture_output=True, text=True)
    return result.stdout
```

I wanted not only rely on publicly available sources, so I decided to also bruteforce DNS-lookups to get some subdomains.
I used dnsx for the DNS-requests because the tool is fast and therefore suitable for bruteforcing.
Since this only queries DNS-Servers and not interact with the target-domains, it is still considered passive reconnaissance.
Considering the runtime of the script,I chose the SecLists 'subdomains-top1million-110000.txt' for bruteforcing. A Larger wordlist would have resulted in way too long scantimes.

I also wrote a python function for automation:

```
def run_dnsx(domain, wordlist):
    cmd = [
        "dnsx",
        "-d", domain,
        "-w", wordlist,
        "-r", resolvers,
        "-t", "200",
        "-resp"
    ]
    result = subprocess.run(cmd, capture_output=True, text=True)
    return result.stdout.splitlines()
```

This is the combined script:

```
import subprocess
import re


def run_dnsx(domain, wordlist):
    cmd = [
        "dnsx",
        "-d", domain,
        "-w", wordlist,
        "-r", resolvers,
        "-t", "200",
        "-resp"
    ]
    result = subprocess.run(cmd, capture_output=True, text=True)
    return result.stdout.splitlines()

def run_theharvester(domain):
    cmd = [
        "theHarvester",
        "-d", domain,
        "-b", "virustotal",
    ]
    result = subprocess.run(cmd, capture_output=True, text=True)
    return result.stdout


dlist_path = ""
with open("domain_list.txt", "r") as f:
    domains1 = f.readlines()
```

```
domains = []
for line in domains1:
    domains.append(line.strip())
wordlist = "/usr/share/wordlists/SecLists/Discovery/DNS/subdomains-
top1million-110000.txt"
resolvers = "resolvers.txt"

for domain in domains:
    print(f"Scanning {domain}...")
    subdomains = run_dnsx(domain, wordlist)
    for sub in subdomains:
        print(sub)
        with open("subdomain_results.txt", "a") as f:
            f.write(sub+"\n")
    print("***Scan with harvester...")
    result = run_theharvester(domain)
    hosts = re.findall(r"([a-zA-Z0-9.-]+\." + re.escape(domain) + r")",
result)

    for sub in hosts:
        print(sub)
        with open("subdomain_results.txt", "a") as f:
            f.write(sub+"\n")
```

Every subdomain has to be resolvable and only counts if they have a A, CNAME, MX or TXT record.
To make sure I only include valid entries in my list I made a script to validate each entry:

```
import subprocess

input = "subdomain_results4.txt"
output = "subdomains3.txt"

cmd = [
    "dnsx",
    "-l", input,
    "-a",
    "-cname",
    "-mx",
    "-txt",
    "-resp",
]


result = subprocess.run(cmd, capture_output=True, text=True)
```

```
valid_subdomains = []


for line in result.stdout.splitlines():
    if not line.strip():
        continue
    subdomain = line.split()[0]
    if subdomain not in valid_subdomains:
        valid_subdomains.append(subdomain)


with open(output, "w") as f:
    for sub in valid_subdomains:
        f.write(sub + "\n")

print(f"Done! {len(valid_subdomains)} valid subdomains written to {output}")
```

This script also makes sure, that every entry is unique.

The virustotal lookup gave around 5,000 subdomains. The bruteforcing resolved almost 80,000 subdomains, but with many duplicates.

After running the validating-script I ended up with 2,257 unique, resolvable subdomains with either an A, CNAME, MX or TXT-Record.

Part 2: Active recon on a specific target

Executive summary:

The next step is to switch from passive to active reconnaissance and from different to a specific target.
The target is a subdomain of the rootdomain cybersteps.de and contains the word "scan".
When the host indentified, a full in-depth reconnaissance should be performed.

1. Target Identification:

The first step is to find the correct subdomain. Before I ran any script I tried the obvious "scanme.cybersteps.de" and it was actually online.
That was of course very lucky, but since this must be the target, I didn't need to run any scripts. But I prob ably would have used ffuf for that.

Running 'nslookup' I also got the corresoonding IP-address: 165.232.131.154

2. Port Scan Results:

I scanned for open TCP-ports using nmap:

```
nmap -sV 165.232.131.154
```

Open TCP ports (with Versions):

Port	Service	Version
21	ftp	vsftpd (before 2.0.0) or WU-FTPD
22	ssh	OpenSSH 8.9p1 Ubuntu 3ubuntu0.13
23	telnet	
80	http	Apache httpd 2.4.6
445	microsoft-ds	
1080	socks5	password protected
3389	ms-wbt-server	
5900	vnc	VNC (protocol 3.8)
9200	sssl/wap-wsp?	

I then scanned for UDP-ports:

```
nmap -sU --top-ports 100 -T4 165.232.131.154
```

Since UDP-scans tend to take long, I decided to add the -T4-flag.

Open or filtered UDP ports:

Port	Status	Service
9	open/filtered	discard
67	open/filtered	dhcps
123	open/filtered	ntp
136	open/filtered	profile
514	open/filtered	syslog
631	open/filtered	ipp
2048	open/filtered	dls-monitor

Port	Status	Service
2222	open/filtered	msantipiracy
4444	open/filtered	krb524
49190	open/filtered	unknown
49194	open/filtered	unknown

3. Service Enumeration & Analysis

port 21 - ftp:

First I tried to log into the ftp-service with anonymous/anonymous (user/pw).
It accepted the username, but not the password.

Then I ran the following command to check for ftp vulnerabilites:

```
nmap -p21 --script ftp-anon,ftp-vsftpd-backdoor,ftp-proftpd-backdoor,ftp* -v
scanme.cybersteps.de
```

This provided me with a valid username and password: test, test

port 80 - HTTP

This command gives me the details about the HTTP-Server.

```
curl -I http://scanme.cybersteps.de
```

It is running Apache 2.4.6.

Neither gobuster or fuff could reveal any directories. All I get back is statuscode 500. I suspect the HTTP-server is not working properly.

Running a vuln-script-scan with nmap revealed a bunch of vulnerabilities.

```
nmap -p80 --script=http-vuln* -sV scanme.cybersteps.de
```

port 23 - Telnet

I was not able to find any easy to guess credentials, but telnet as a whole is not safe.

port 445 - SMB

I ran this script to try to enumerate the SMB-Service.

```
nmap -p445 --script=smb-enum*,smb-vuln* scanme.cybersteps.de
```

Enumeration did not work, which indicates the service is restricted and hardened.

port 1080 - socks5

Socks is a proxy service.
This command tells me, that the proxy is not open and will not allow relay traffic from anyone:

```
nmap -p1080 --script=socks-open-proxy scanme.cybersteps.de
```

port 3389 - RDP and port 5900 - VNC

RDP and VNC are both remote desktop protocols.
VNC could be unencrypted and therefore a target for credential-sniffing.
If RDP is unpatched it could be vulnerable.

4. Security Issues & Vulnerability Analysis

1. The FTP-Service has an eassily guessed username/password with test/test. It is also an outdated Version, which is potentially vulnerable to a backdoor attack.

Improvements:

Use a newer FTP-Service (Ideally SFTP) or HTTPS-based filetransfer and check for weak credentials like test/test.

2. The HTTP-Server Apache 2.4.6 is very outdated and has multiple severe vulnerabilities.

Improvements:

Update to a newer version.

3. Telnet as a service is outdated and should be avoided. It is considered unsafe because of the unencrypted data and credentials-transfer. It is also redundant, because SSH is also enabled.

Improvements:

Disable Telnet and use SSH instead.

4. Using VNC and RDP is redundant, as both services are remote desktop protocols. Exposing a remote desktop service to the internet is always risky, but having both doubles the attack-vector.

Improvements:

Choose one of these services, if really necessary. When choosing VNC enable encryption. Be sure these services are updated and do not use easy to guess/bruteforce credentials.

5. SMB should not be exposed to the internet. It is a service with many severe vulnerabilities like EternalBlue.

Improvements:

Either remove or firewall the SMB-service.